

▼ Módulo 1: Variáveis e Memória

Certifique-se de que você está dentro da pasta da aula antes de executar o comando abaixo. O comando criará automaticamente as subpastas `cpp/src`, `cpp/bin` e `cpp/lib` dentro dela.

```
!mkdir -p cpp/src cpp/bin cpp/lib
```

A estrutura gerada será a seguinte.

```
NOME_DA_AULA/  
├── cpp/  
│   ├── src/  
│   ├── bin/  
│   └── lib/
```

Pasta	Conteúdo	Extensões
<code>src</code>	Arquivos fonte com a lógica do programa	<code>.cpp</code>
<code>bin</code>	Executáveis gerados pelo compilador	sem extensão
<code>lib</code>	Cabeçalhos e bibliotecas externas	<code>.h</code> <code>.hpp</code>

Nota sobre os comandos do Jupyter Notebook

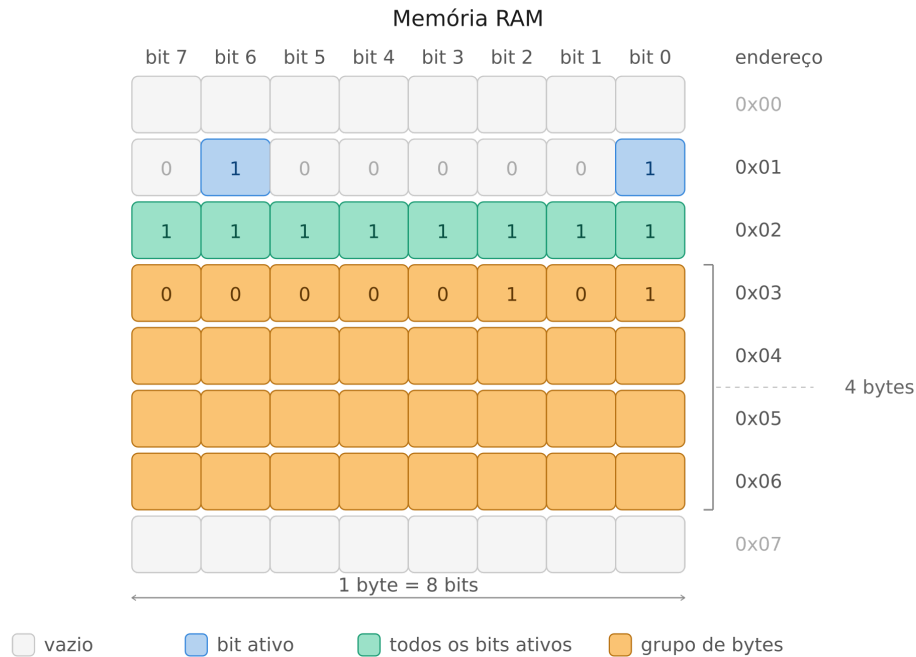
Você vai notar que os exemplos desta aula utilizam alguns comandos especiais, como `%%writefile`, `!g++` e `!./...`. É importante entender que **esses prefixos não fazem parte do C++**. O `%%writefile` e o `!` são apenas uma "ponte" entre o Jupyter e o terminal do sistema. **O fluxo real é o mesmo que você vê nos vídeos:** escrever o código, compilar com g++ e executar o binário. — eles são recursos do próprio Jupyter Notebook que nos permitem escrever e executar código C++ diretamente por aqui.

- `%%writefile cpp/src/02_exec_02.cpp` salva o conteúdo da célula como um arquivo `.cpp` no disco.
- `!g++ cpp/src/02_exec_02.cpp -o cpp/bin/02_exec_02` compila o arquivo usando o g++, exatamente como você faria no terminal.
- `!./cpp/bin/02_exec_02` executa o binário gerado.

Disponibilizamos os exemplos dessa forma para que você consiga acompanhar e rodar tudo diretamente na aula, sem precisar sair do notebook, de modo online e interativo. Mas recomendamos que, sempre que possível, **pratique também pelo terminal**, como demonstrado nos vídeos — é assim que você vai trabalhar no dia a dia com C++.

▼ 1.1 Variáveis e Memória

Para programar em C++ com eficiência, é preciso entender o que acontece por baixo do código, na memória RAM. A memória é o local físico onde o programa armazena todos os dados enquanto está em execução. Cada vez que você cria uma variável, o programa está reservando um pedaço dessa memória para guardar um valor.



A memória é organizada em unidades muito pequenas. A menor delas é o **bit** (binary digit), que armazena um único valor, 0 ou 1. Sozinho, um bit não carrega muita informação útil. Por isso, o computador agrupa bits em conjuntos maiores. O agrupamento fundamental é o **byte**, composto por 8 bits consecutivos. Com 8 bits, conseguimos representar $2^8 = 256$ combinações diferentes, o que já permite armazenar um caractere da tabela ASCII ou um número inteiro de 0 a 255. Dependendo do tipo de dado que queremos armazenar, o C++ pode agrupar 1, 2, 4 ou até 8 bytes para formar uma única informação. Quanto mais bytes, maior o intervalo de valores que a variável consegue representar.

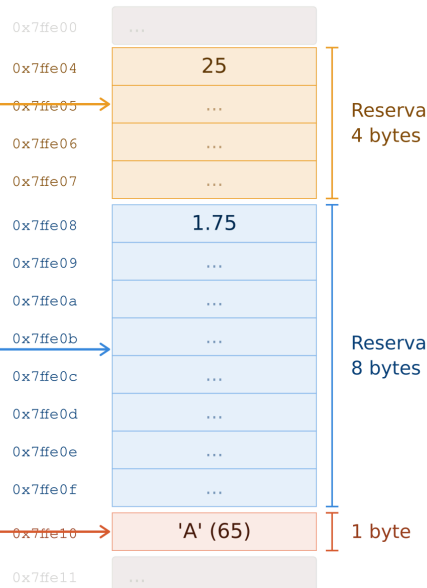
Agrupamento	Bits	Combinações possíveis
1 byte	8 bits	256
2 bytes	16 bits	65.536
4 bytes	32 bits	~4,2 bilhões
8 bytes	64 bits	~18,4 quintilhões

Com isso em mente, observe a imagem abaixo. Do lado esquerdo temos três declarações em C++, e do lado direito a memória RAM com os endereços e os bytes que cada variável reserva.

Código C++

```
int idade = 25;
double altura = 1.75;
char letra = 'A';
```

Memória RAM

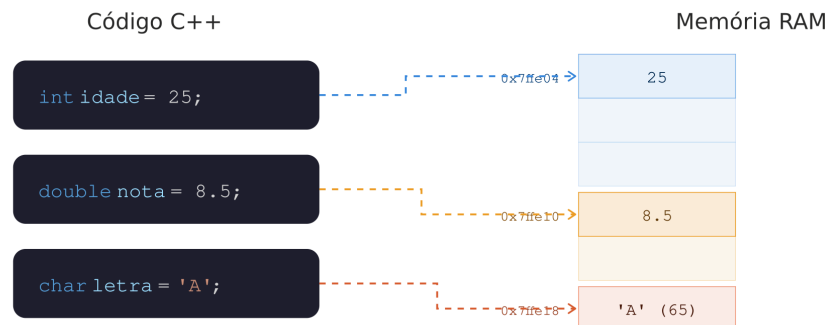


O `int idade` reserva 4 bytes (32 bits, cerca de 4,2 bilhões de combinações). O `double altura` reserva 8 bytes (64 bits, cerca de 18,4 quintilhões de combinações). O `char letra` reserva apenas 1 byte (8 bits, 256 combinações). Cada tipo existe justamente para dizer ao compilador quantos bytes reservar e como interpretar os bits que serão guardados ali.

1.1.1 O que é uma Variável

Uma variável é, em essência, um pedaço nomeado de memória que reservamos para armazenar um tipo específico de dado. Ao criar uma variável, você está dizendo ao sistema algo como "separe 4 bytes de memória para mim e dê a eles o nome `idade`, pois vou guardar um número inteiro aqui."

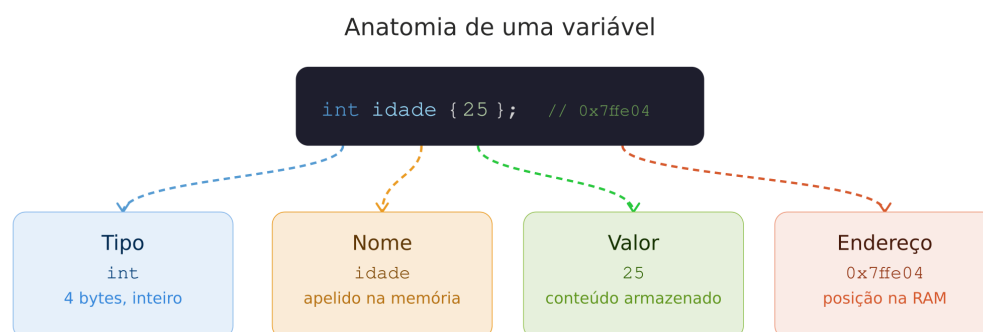
Na memória física, os dados são acessados por endereços numéricos representados em hexadecimal, como `0x7ffe04`. Seria impossível programar se tivéssemos que memorizar endereços como esse para cada pedaço de informação. A variável resolve esse problema, ela funciona como um apelido para o endereço. O programador usa nomes significativos como `idade`, `nota`, `letra`, e o compilador se encarrega de traduzir esses nomes para os endereços reais na memória. A imagem abaixo ilustra exatamente esse processo.



Observe que no código escrevemos apenas `int idade = 25`, mas por baixo o compilador associa esse nome ao endereço `0x7ffe04` na RAM. O programador nunca precisa lidar com esses endereços diretamente.

1.1.2 Anatomia de uma Variável

Toda variável possui quatro componentes que definem sua existência na memória. A imagem abaixo decompõe a declaração `int idade {25};` nesses quatro elementos.



1. Tipo: Define quantos bytes a variável ocupa na memória e como o computador deve interpretar os bits armazenados. Sem o tipo, os dados são apenas zeros e uns sem significado. No exemplo acima, `int` informa ao compilador que deve reservar 4 bytes e interpretá-los como número inteiro.

```
// A mesma sequência binária 01000001 pode ser interpretada como:
int numero {65};    // o número inteiro 65
char letra {65};    // o caractere 'A' na tabela ASCII
```

A sequência `01000001` é idêntica nos dois casos. O que muda é como o compilador interpreta esse padrão de bits, e isso é definido exclusivamente pelo tipo.

2. Nome (Identif): É o rótulo que usamos no código para nos referir àquele pedaço de memória. O C++ é *case-sensitive*, o que significa que `Idade`, `idade` e `IDADE` são três variáveis distintas. O nome não pode começar com números e não pode usar palavras reservadas da linguagem, como `int`, `return` ou `while`.

```
int idade {22};      // válido
int _contador {0};  // válido, pode começar com underscore
// int 2valor {10}; // inválido, começa com número
```

3. Valor (Value): É o conteúdo armazenado naquela posição de memória. Do ponto de vista da máquina, o valor é estritamente uma sequência de zeros e uns. Do ponto de vista do programador, o computador traduz essa sequência binária para a representação humana automaticamente quando acessamos a variável.

4. Endereço (Address): É o local físico na memória onde a sequência de bits começa. Cada variável ocupa um endereço único durante seu tempo de vida. Não precisamos conhecer esse endereço diretamente para programar, mas ele existe e pode ser acessado quando necessário em tópicos mais avançados.

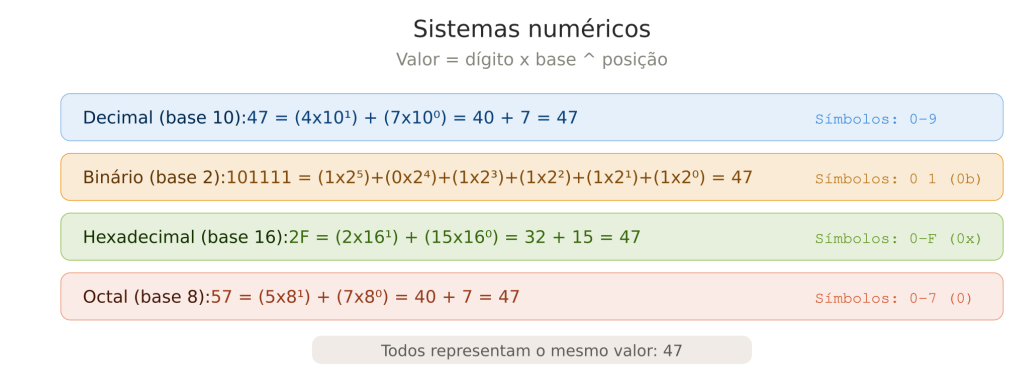
Em C++, as variáveis devem ser declaradas antes de serem usadas. Isso é necessário para que o compilador saiba exatamente quantos bytes reservar na memória e como interpretar os bits que serão colocados lá.

1.2 Sistemas Numéricos

Antes de declararmos variáveis, é importante entender como os dados numéricos são representados no nível mais fundamental. O computador trabalha exclusivamente com estados de ligado e desligado (0 e 1), mas o C++ nos oferece diferentes formas de escrever esses valores no código. A regra que conecta todas essas formas é a mesma da matemática. O valor de um número em qualquer base é calculado pela soma de potências dessa base.

$$Valor = \sum (dígito \times base^{posição})$$

A imagem abaixo aplica essa fórmula ao número 47 nos quatro sistemas que o C++ suporta. Observe que o resultado final é sempre o mesmo, o que muda é a base e os símbolos usados para representá-lo.



1.2.1 Decimal (Base 10)

É o sistema que usamos no dia a dia. Possui dez símbolos, de 0 a 9, e cada posição vale dez vezes a anterior. Como visto na imagem, o número 47 é decomposto em $(4 \times 10^1) + (7 \times 10^0) = 40 + 7$. Para um número maior, a lógica é a mesma, basta expandir as posições.

$$2371 = (2 \times 10^3) + (3 \times 10^2) + (7 \times 10^1) + (1 \times 10^0) = 2000 + 300 + 70 + 1$$

Em C++, quando escrevemos um número sem nenhum prefixo, ele é interpretado como decimal.

```
int valor {47}; // decimal, o padrão
int preco {150}; // decimal, sem prefixo
```

1.2.2 Binário (Base 2)

É a representação nativa da máquina. Utiliza apenas dois símbolos, 0 e 1, e cada posição é uma potência de 2. Na imagem, o mesmo 47 aparece como 101111, que é a soma $32+0+8+4+2+1$. A fórmula expandida para um outro exemplo fica assim.

$$100101_2 = (1 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 32 + 4 + 1 = 37_{10}$$

Em C++, o prefixo 0b indica que o número está em binário (disponível a partir do C++14).

```
int flags {0b101111}; // binário, equivale a 47 em decimal
int mascara {0b1010}; // binário, equivale a 10 em decimal
```

O binário é usado frequentemente para máscaras de bits (bitmasks), onde cada bit individual representa uma opção ligada ou desligada.

1.2.3 Hexadecimal (Base 16)

O hexadecimal é o sistema preferido para visualizar memória e endereços. Ele condensa o binário de forma compacta, pois cada dígito hexadecimal corresponde exatamente a 4 bits. Os símbolos vão de 0 a 9 e de A a F, conforme a tabela abaixo.

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Na imagem, 47 é representado como 2F, que resulta em $(2 \times 16^1) + (15 \times 16^0) = 32 + 15$. A fórmula expandida para um exemplo com mais dígitos segue o mesmo padrão.

$$1A3_{16} = (1 \times 16^2) + (10 \times 16^1) + (3 \times 16^0) = 256 + 160 + 3 = 419_{10}$$

O prefixo em C++ é `0x`.

```
int endereco {0x2F}; // hexadecimal, equivale a 47 em decimal
int cor {0xFF};      // hexadecimal, equivale a 255 em decimal
```

1.2.4 Octal (Base 8)

O sistema octal usa os símbolos de 0 a 7 e é um legado de computadores mais antigos. Na imagem, 47 aparece como `57` em octal, resultando em $(5 \times 8^1) + (7 \times 8^0) = 40 + 7$. A fórmula expandida para um número maior segue a mesma regra.

$$327_8 = (3 \times 8^2) + (2 \times 8^1) + (7 \times 8^0) = 192 + 16 + 7 = 215_{10}$$

Raramente é usado em código moderno, mas é importante conhecê-lo porque o C++ interpreta qualquer número que comece com zero como octal. O prefixo é simplesmente `0`.

```
int valor_octal {057}; // octal, equivale a 47 em decimal
int armadilha {020};  // CUIDADO: octal, equivale a 16, NÃO a 20
```

Esse comportamento é uma armadilha clássica. Se um programador escreve `020` pensando no número 20, o resultado será 16. Nunca comece um número decimal com zero a menos que a intenção seja realmente usar o sistema octal.

O ponto central, como a imagem resume na parte inferior, é que todas as representações armazenam exatamente o mesmo valor na memória. Decimal, binário, hexadecimal e octal são apenas formas diferentes de escrever a mesma sequência de bits. O `cout` sempre exibe o valor em decimal por padrão, independentemente de como o número foi escrito no código. O programa abaixo demonstra que o mesmo valor (15 em decimal) pode ser declarado em quatro sistemas diferentes e produz a mesma saída.

Exemplo 1: Sistemas numéricos

```
%writefile cpp/src/01_exec_01.cpp
#include <iostream>

using namespace std;

int main(){

    int number1 = 15;           // Decimal
    int number2 = 017;          // Octal
    int number3 = 0x0f;         // Hexadecimal
    int number4 = 0b00001111;   // Binary - C++14

    cout << "Sistema Decimal" << endl;
    cout << "Number1 é: " << number1 << endl ;
    cout << "Number2 é: " << number2 << endl ;
    cout << "Number3 é: " << number3 << endl ;
    cout << "Number4 é: " << number4 << endl ;

    return 0;

}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/01_exec_01.cpp -o cpp/bin/01_exec_01
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/01_exec_01`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/01_exec_01
```

✓ 1.3 Declaração e Inicialização de Variáveis

Declarar uma variável é o ato de informar ao compilador o tipo e o nome daquele pedaço de memória. Inicializar é atribuir um valor a ela no momento da criação. O C++ oferece três formas de fazer isso, e a escolha entre elas não é apenas uma questão de estilo, cada forma possui um nível diferente de segurança.

1.3.1 Inicialização por Chaves (Braced Initialization)

Esta é a forma recomendada em C++ moderno. A sintaxe utiliza chaves `{}` após o nome da variável.

```
int contador {};    // inicializa com zero automaticamente
int limite {100};  // inicializa com o valor 100
```

AA grande vantagem dessa forma é a proteção contra conversões de estreitamento (ou **narrowing conversions**), ou seja, perda de dados por conversão de tipo. Se tentarmos atribuir um valor `double` a uma variável `int`, o compilador gera um erro.

```
// int resultado {3.7}; // ERRO: narrowing conversion
// 3.7 é double, não cabe em int sem perda
```

Esse comportamento é intencional e desejável. O compilador está nos protegendo de uma perda de informação silenciosa.

Outra vantagem é que uma variável inicializada com `{}` vazio recebe automaticamente o valor zero. Isso evita o problema clássico de variáveis contendo lixo de memória.

```
int total {}; // total vale 0, seguro e previsível
```

Também é possível usar expressões dentro das chaves.

```
int largura {10};
int altura {5};
int area {largura * altura}; // area recebe 50
```

1.3.2 Inicialização Funcional

Usa parênteses e se assemelha a uma chamada de função. É válida, mas menos segura que a inicialização por chaves.

```
int peso (75);    // inicializa com 75
int volume (10);  // inicializa com 10
```

O problema é que esta forma aceita conversões com perda de dados sem gerar erro.

```
int truncado (3.7); // COMPILA sem erro, truncado recebe 3
// a parte decimal é descartada silenciosamente
```

Isso pode introduzir bugs difíceis de detectar, especialmente em programas maiores onde os tipos nem sempre são óbvios.

1.3.3 Inicialização por Atribuição

É a forma herdada da linguagem C e a mais familiar para quem vem de outras linguagens. Utiliza o operador `=`.

```
int distancia = 42;
int velocidade = 80;
```

Assim como a forma funcional, a inicialização por atribuição também permite conversão com perda de dados silenciosa.

```
int arredondado = 9.8; // COMPILA sem erro, arredondado recebe 9
```

Além disso, é possível declarar uma variável sem atribuir valor é permitido pelo compilador, mas extremamente perigoso.

```
int lixo;
// lixo contém um valor imprevisível, o que estava na memória antes
```

O valor de `lixo` será qualquer sequência de bits que já estivesse naquela posição da memória. Isso pode ser 0, 42, -1987 ou qualquer outro número. O comportamento é indefinido e pode mudar entre execuções do programa. Sempre inicialize suas variáveis.

1.3.4 O operador sizeof

O operador `sizeof` retorna o número de bytes que um tipo ou uma variável ocupa na memória. Ele é útil para verificar o tamanho real dos tipos na sua plataforma.

```
int idade {25};
cout << sizeof(int) << endl;    // imprime 4 (bytes)
cout << sizeof(idade) << endl;  // imprime 4 (bytes)
```

O programa abaixo demonstra as três formas de inicialização, incluindo o comportamento de narrowing conversion em cada uma, e o uso de `sizeof` para verificar o tamanho das variáveis.

Exemplo 2: Declaração e inicialização

```

%%writefile cpp/src/01_exec_02.cpp
#include <iostream>

using namespace std;

int main(){

    /* Inicialização por Chaves */

    // Variável inicializada sem valor, contém qualquer valor alocado pelo sistema -> Gera um WARNING
    int elephant_count;

    int lion_count {}; // Inicializada com ZERO.

    int dog_count {10}; // Inicializada com 10.

    int cat_count {15}; // Inicializada com 15

    // Podemos usar expressões para inicialização
    int domesticated_animals {dog_count + cat_count};

    // 2.9 é do tipo double, que é outro tipo de dado -> ERRO ou WARNING.
    // int narrowing_conversion {2.9};

    cout << "Contagem de elefantes: " << elephant_count << endl;
    cout << "Contagem de leões: " << lion_count << endl;
    cout << "Contagem de cães: " << dog_count << endl;
    cout << "Contagem de gatos: " << cat_count << endl;
    cout << "Contagem de animais domésticos: " << domesticated_animals << endl;
    cout << "-----" << endl;

    /* Inicialização de Variável Funcional */

    int apple_count (10); // Inicializada com 10.

    int orange_count (10); // Inicializada com 10

    int fruit_count (apple_count + orange_count);

    // int bad_initialization (doesnt_exist_3 + doesnt_exist4);

    // Perda de Informação, menos seguro que a anterior -> Ele inicializará a variável com 2.
    int narrowing_conversion_functional (2.9);

    cout << "Contagem de maçãs: " << apple_count << endl;
    cout << "Contagem de laranjas: " << orange_count << endl;
    cout << "Contagem de frutas: " << fruit_count << endl;
    cout << "Conversão de estreitamento: " << narrowing_conversion_functional << endl;
    cout << "-----" << endl;

    /* Inicialização por Atribuição */

    int bike_count = 2;

    int truck_count = 7;

    int vehicle_count = bike_count + truck_count;

    int narrowing_conversation_assignment = 2.9;

    cout << "Contagem de bicicletas: " << bike_count << endl;
    cout << "Contagem de caminhões: " << truck_count << endl;
    cout << "Contagem de veículos: " << vehicle_count << endl;
    cout << "Conversão de estreitamento: " << narrowing_conversation_assignment << endl;
    cout << "-----" << endl;

    cout << "sizeof int: " << sizeof(int) << endl;
    cout << "sizeof truck_count: " << sizeof(truck_count) << endl;
    return 0;
}

```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/01_exec_02.cpp -o cpp/bin/01_exec_02
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/01_exec_02`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

!./cpp/bin/01_exec_02

1.4 Tipos Inteiros

Os tipos inteiros armazenam números sem parte fracionária. O tipo mais comum é o `int`, que tipicamente ocupa 4 bytes (32 bits) na maioria das plataformas modernas.

1.4.1 Tipos e Tamanhos

O C++ define vários tipos inteiros com tamanhos mínimos garantidos pelo padrão. Na prática, os tamanhos mais comuns em arquiteturas x86-64 são os seguintes.

Tipo	Tamanho típico	Intervalo (com sinal)
<code>short</code>	2 bytes	-32.768 a 32.767
<code>int</code>	4 bytes	-2.147.483.648 a 2.147.483.647
<code>long</code>	4 ou 8 bytes	depende da plataforma
<code>long long</code>	8 bytes	-9,2 x 10 ¹⁸ a 9,2 x 10 ¹⁸

O `int` é o tipo inteiro padrão e deve ser a primeira escolha na maioria dos casos. Seus 4 bytes permitem representar mais de 4 bilhões de valores distintos, o que é suficiente para a grande maioria das aplicações.

1.4.2 Modificadores de Sinal

Cada tipo inteiro pode receber o modificador `signed` ou `unsigned`. O modificador `signed` é o padrão, ou seja, `int` e `signed int` são a mesma coisa. Ele permite representar números negativos e positivos. O modificador `unsigned` elimina os valores negativos e dobra a capacidade no lado positivo.

```
int saldo {-500};           // signed por padrão, aceita negativos
unsigned int populacao {300000}; // apenas positivos, de 0 a ~4,2 bilhões
```

A tabela abaixo mostra como o modificador `unsigned` altera o intervalo do tipo `int` (4 bytes).

Tipo	Mínimo	Máximo
<code>int</code> (signed)	-2.147.483.648	2.147.483.647
<code>unsigned int</code>	0	4.294.967.295

O número total de combinações não muda (ambos representam 2³² valores). O que muda é como essas combinações são distribuídas. O `signed` divide entre negativos e positivos. O `unsigned` concentra tudo no lado positivo.

1.4.3 Inteiros de Largura Fixa

Um problema dos tipos padrão como `int` e `long` é que o tamanho exato pode variar de plataforma para plataforma. O padrão C++ garante apenas tamanhos mínimos, não exatos. Para resolver isso, o C++11 introduziu os inteiros de largura fixa, definidos no header `<cstdint>`. Esses tipos garantem exatamente o mesmo tamanho em qualquer arquitetura.

Tipo	Tamanho garantido	Intervalo
<code>std::int8_t</code>	1 byte com sinal	-128 a 127
<code>std::uint8_t</code>	1 byte sem sinal	0 a 255
<code>std::int16_t</code>	2 bytes com sinal	-32.768 a 32.767
<code>std::uint16_t</code>	2 bytes sem sinal	0 a 65.535
<code>std::int32_t</code>	4 bytes com sinal	-2.147.483.648 a 2.147.483.647
<code>std::uint32_t</code>	4 bytes sem sinal	0 a 4.294.967.295
<code>std::int64_t</code>	8 bytes com sinal	-9,2 x 10 ¹⁸ a 9,2 x 10 ¹⁸
<code>std::uint64_t</code>	8 bytes sem sinal	0 a 18,4 x 10 ¹⁸

Para usar esses tipos, é necessário incluir o header `<cstdint>`.

```
# include <cstdint>

std::int32_t sensor_id {42000}; // sempre 4 bytes, em qualquer plataforma
std::uint8_t canal {200};       // sempre 1 byte, sem sinal
```

Um detalhe importante é que `std::int8_t` e `std::uint8_t` são, na prática, o mesmo tipo que `signed char` e `unsigned char`. Isso aconteceu por conveniência. O `char` já existia na linguagem desde o C original e já ocupava exatamente 1 byte (8 bits). Quando o padrão C99 precisou definir um tipo inteiro de exatamente 8 bits, o `signed char` já atendia todos os requisitos, mesma quantidade de bits, mesma representação em memória. Não fazia sentido inventar um tipo novo, então os compiladores simplesmente definiram `int8_t` como sendo `signed char` por baixo. O efeito colateral é que o `cout`, que já existia antes do `int8_t` ser criado, trata qualquer variação de `char` como caractere. Quando o `int8_t` chegou disfarçado de `signed char`, o `cout` não tinha como distinguir se o programador queria imprimir um número ou uma letra. A biblioteca `iostream` nunca foi atualizada para corrigir isso.


```
std::int8_t valor {65};
cout << valor << endl;    // imprime A, não 65
```

1.5 Tipos de Ponto Flutuante

Quando precisamos representar números com parte fracionária, usamos os tipos de ponto flutuante. O C++ oferece três tipos, cada um com um nível diferente de precisão e tamanho na memória.

Tipo	Tamanho	Precisão aproximada	Sufixo
<code>float</code>	4 bytes	~7 dígitos significativos	<code>f</code>
<code>double</code>	8 bytes	~15 dígitos significativos	nenhum (padrão)
<code>long double</code>	12 ou 16 bytes	~18 dígitos significativos	<code>L</code>

O tipo `double` é o padrão recomendado para a maioria das aplicações. Quando escrevemos um número com ponto decimal no código, como `3.14`, o compilador assume automaticamente que é um `double`. Para indicar que um literal é `float`, adicionamos o sufixo `f`. Para `long double`, usamos o sufixo `L`.

```
float temperatura {36.5f};           // float, 4 bytes
double velocidade {299792458.0};      // double, 8 bytes (padrão)
long double pi {3.14159265358979323846L}; // long double
```

Por que usar os sufixos? Sem o sufixo, qualquer número decimal como `36.5` é tratado como `double`. Ao escrever `float x {36.5}`, o compilador primeiro cria um `double`, depois converte para `float`, podendo perder precisão no caminho. Com o sufixo `f`, o literal `36.5f` já nasce como `float`, sem conversão intermediária. A diferença não aparece na saída padrão do `cout`, mas fica visível quando imprimimos com `setprecision(20)`, onde os dígitos que divergem entre `float` e `double` ficam expostos.

Sobre o tamanho do `long double`. A variação entre 12 e 16 bytes existe porque os 10 bytes úteis de precisão recebem bytes extras de preenchimento para manter o alinhamento de memória. No GCC em Linux 32 bits, o tamanho alocado é 12 bytes. Em sistemas 64 bits, sobe para 16 bytes. Em ambos os casos, a precisão real é a mesma. O caso mais importante para saber é que no MSVC (Visual Studio no Windows), `long double` é idêntico ao `double` (8 bytes), sem precisão extra nenhuma.

1.5.1 Precisão

A precisão de um tipo de ponto flutuante indica quantos dígitos significativos ele consegue representar com exatidão. Além desse limite, os dígitos restantes são aproximações. Isso acontece porque o computador dispõe de uma quantidade fixa de bits para representar cada número. Um `float` usa 32 bits e consegue representar com fidelidade cerca de 7 dígitos. Um `double` usa 64 bits e mantém cerca de 15 dígitos confiáveis. Qualquer dígito além desse limite é ruído, ou seja, o valor armazenado na memória não corresponde exatamente ao que foi escrito no código.

```
float f {1.123456789012345f};        // apenas os 7 primeiros dígitos são confiáveis
double d {1.123456789012345000};      // os 15 primeiros dígitos são confiáveis
```

Para visualizar a precisão real de cada tipo, podemos usar `std::setprecision()` da biblioteca `<iomanip>`, que controla quantos dígitos o `cout` exibe.

```
# include <iomanip>
cout << setprecision(20);
cout << f << endl; // mostra que após o 7º dígito, os valores divergem
cout << d << endl; // mantém fidelidade até o 15º dígito aproximadamente
```

1.5.2 Notação Científica

Para números muito grandes ou muito pequenos, o C++ suporta notação científica usando a letra `e` seguida do expoente. A notação fixa é a forma convencional de escrever o número com todos os dígitos, como `0.00000000003498`. A notação científica representa o mesmo valor de forma compacta, como `3.498e-11`, que significa 3.498×10^{-11} .

```
double massa_terra {5.972e24};        // 5.972 x 10^24
double carga_eletron {1.602e-19};     // 1.602 x 10^(-19)
```

Ambas as formas, fixa e científica, produzem o mesmo valor na memória. A notação científica é apenas uma conveniência para evitar escrever dezenas de zeros.

1.5.3 Infinity e NaN

O padrão IEEE 754 define dois valores especiais que podem surgir em operações de ponto flutuante.

Infinity. Quando dividimos um número de ponto flutuante por zero, o resultado é infinito positivo ou negativo.

```
double positivo {5.0};
double zero {0.0};
double resultado = positivo / zero; // resultado é +inf
```

NaN (Not a Number). Quando realizamos uma operação matematicamente indefinida, como dividir zero por zero, o resultado é **NaN**.

```
double zero1 {0.0};
double zero2 {0.0};
double resultado = zero1 / zero2; // resultado é NaN (nan)
```

Esses valores não geram erros de execução, o programa continua rodando. Porém, qualquer operação envolvendo **NaN** sempre resulta em **NaN**, e qualquer operação envolvendo **Infinity** propaga o infinito. O programa abaixo demonstra a precisão de cada tipo, a notação científica e os valores especiais.

Exemplo 3: Ponto flutuante, precisão e valores especiais

```
%writefile cpp/src/01_exec_03.cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main(){

    float number1 {1.12345678901234567890f}; // Precisão: 7
    double number2 {1.12345678901234567890}; // Precisão: 15
    long double number3 {1.12345678901234567890L};

    // Printando os tamanhos
    cout << "sizeof float: " << sizeof(float) << endl;
    cout << "sizeof double: " << sizeof(double) << endl;
    cout << "sizeof long double: " << sizeof(long double) << endl;

    // Precisão
    cout << setprecision(20); // Controla a precisão do std::cout
    cout << "number1 é: " << number1 << endl; // 7 dígitos
    cout << "number2 é: " << number2 << endl; // 15 dígitos
    cout << "number3 é: " << number3 << endl; // 15+ dígitos

    // Problemas com o tipo Float: A precisão é usualmente muito limitada
    // Para muitas aplicações
    double number4 {192400023.0};

    cout << "number4 é: " << number4 << endl;

    // Notação Científica

    cout << "-----" << endl;

    double number5 {192400023};
    double number6 {1.92400023e8};
    double number7 {1.924e8};
    double number8 {0.00000000003498};
    double number9 {3.498e-11};

    cout << "number5 é: " << number5 << endl;
    cout << "number6 é: " << number6 << endl;
    cout << "number7 é: " << number7 << endl;
    cout << "number8 é: " << number8 << endl;
    cout << "number9 é: " << number9 << endl;

    // Infinito e NaN

    cout << endl;
    cout << "Infinito e NaN" << endl;

    double number10 {5.6};
    double number11 {}; // Inicializados com 0
    double number12 {}; // Inicializados com 0

    // Infinito
    double result {number10 / number11};

    cout << number10 << "/" << number11 << " resulta em " << result << endl;
    cout << result << " + " << number10 << " resulta em " << result + number10 << endl;

    // NaN
    result = number11 / number12;
```

```

    cout << number11 << "/" << number12 << " = " << result << endl;

    return 0;
}

```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/01_exec_03.cpp -o cpp/bin/01_exec_03
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/01_exec_03`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/01_exec_03
```

✓ 1.6 Booleanos

O tipo `bool` representa apenas dois estados possíveis, verdadeiro (`true`) ou falso (`false`). Internamente, `true` é armazenado como 1 e `false` como 0. Apesar de precisar de apenas 1 bit para representar dois estados, o `bool` ocupa 1 byte (8 bits) na memória. Isso acontece porque o byte é a menor unidade endereçável na maioria das arquiteturas.

```

bool motor_ligado {true};
bool porta_aberta {false};
cout << sizeof(bool) << endl; // imprime 1 (byte)

```

Por padrão, o `cout` exibe valores booleanos como números, 1 para `true` e 0 para `false`. Para que a saída exiba as palavras `true` e `false`, utilizamos o manipulador `boolalpha`.

```

bool conectado {true};
cout << conectado << endl;           // imprime 1
cout << boolalpha;
cout << conectado << endl;           // imprime true

```

O tipo `bool` é fundamental para expressões condicionais e operadores lógicos, que serão abordados na próxima aula. Neste momento, o mais importante é entender que ele existe, que ocupa 1 byte e que pode assumir apenas dois valores. O programa abaixo demonstra a criação de variáveis booleanas, a impressão padrão e com `boolalpha`, e o tamanho do tipo na memória.

Exemplo 4: Booleanos

```

%%writefile cpp/src/01_exec_04.cpp
#include <iostream>
using namespace std;

int main(){

    bool red_light {true};
    bool green_light {false};

    if(red_light == true){
        cout << "Pare!" << endl;
    }else{
        cout << "Siga em frente!" << endl;
    }

    if(green_light){
        cout << "O sinal está verde!" << endl;
    }else{
        cout << "O sinal NÃO está verde!" << endl;
    }

    cout << "Sizeof(bool): " << sizeof(bool) << endl;

    // Imprimindo um bool
    // 1 --> true
    // 0 --> false
    cout << endl;
    cout << "red_light: " << red_light << endl;
    cout << "green_light: " << green_light << endl;

    cout << boolalpha; // Troca 0 e 1 para true e false
    cout << "red_light: " << red_light << endl;

```

```
cout << "green_light: " << green_light << endl;

return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/01_exec_04.cpp -o cpp/bin/01_exec_04
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/01_exec_04`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/01_exec_04
```

1.7 Caracteres e Texto

O tipo `char` ocupa exatamente 1 byte (8 bits) e é usado para armazenar um único caractere. Internamente, o `char` não armazena a letra em si, ele armazena um número inteiro que corresponde a um caractere na tabela ASCII. Essa tabela define um mapeamento padronizado entre números e caracteres.

Uma breve história. ASCII significa American Standard Code for Information Interchange. Ele foi criado em 1963 pela American Standards Association (hoje ANSI) com o objetivo de padronizar como computadores e equipamentos de telecomunicação representam texto. Antes dele, cada fabricante usava seu próprio código, o que tornava a comunicação entre máquinas diferentes praticamente impossível. A ideia nasceu dos códigos telegráficos, como o código Morse e o código Baudot de 5 bits usado em teletipos. O comitê decidiu usar 7 bits, o que permite 128 caracteres. A primeira versão continha apenas letras maiúsculas e números. Em 1967, as letras minúsculas foram adicionadas, formando o padrão que usamos até hoje. Um detalhe elegante do projeto é que a diferença entre uma letra maiúscula e sua versão minúscula é sempre exatamente 32, ou seja, apenas 1 bit muda na representação binária. Isso não foi acidente, foi projetado assim para simplificar a construção de teclados e impressoras da época.

A tabela abaixo mostra os caracteres imprimíveis do ASCII (códigos 32 a 126). Os códigos de 0 a 31 são caracteres de controle (como tabulação e quebra de linha) e não possuem representação visual.

Dec	Char	Dec	Char	Dec	Char	Dec	Char	Dec	Char	Dec	Char
32	(espaço)	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o		

Alguns padrões que vale a pena observar na tabela. Os dígitos `'0'` a `'9'` ocupam os códigos 48 a 57, em sequência. As letras maiúsculas `'A'` a `'Z'` vão de 65 a 90. As minúsculas `'a'` a `'z'` vão de 97 a 122. A diferença entre maiúscula e minúscula é sempre 32 (`'A'` = 65, `'a'` = 97).

Quando declaramos um `char` com uma letra entre aspas simples, o compilador converte automaticamente para o código ASCII correspondente.

```
char inicial {'M'}; // armazena o valor 77 (código ASCII de 'M')
char digito {'7'}; // armazena o valor 55 (código ASCII de '7')
```

Como o `char` é essencialmente um inteiro de 1 byte, podemos atribuir um número diretamente e ele será interpretado como o caractere correspondente.

```
char letra {65}; // armazena 65, que corresponde a 'A'
cout << letra << endl; // imprime A
```

Isso também significa que podemos fazer aritmética com caracteres. A diferença entre uma letra maiúscula e sua versão minúscula na tabela ASCII é sempre 32. Da mesma forma, subtrair o caractere `'0'` de um dígito nos dá o valor numérico correspondente.

```
char maiuscula {'A'};
char minuscula = maiuscula + 32;    // 65 + 32 = 97, que é 'a'

char digito_char {'5'};
int digito_num = digito_char - '0'; // 53 - 48 = 5 (valor numérico)
```

Quando imprimimos um `char` com o `cout`, ele sempre exibe o caractere, nunca o número. Para ver o valor numérico armazenado internamente, existem técnicas de conversão de tipo que serão abordadas em uma aula dedicada. Por enquanto, o importante é entender que o `char` é um inteiro disfarçado de letra.

```
char c {'Z'};
cout << c << endl;    // imprime Z (o caractere)
int n = c;              // copia o valor numérico para um int
cout << n << endl;    // imprime 90 (o código ASCII)
```

O programa abaixo demonstra a criação de variáveis `char`, a relação entre o caractere e seu valor ASCII, e como acessar o valor numérico copiando para uma variável `int`.

Exemplo 5: Caracteres e ASCII

```
%%writefile cpp/src/01_exec_05.cpp
#include <iostream>

using namespace std;

int main(){

    char character1 {'a'};
    char character2 {'r'};
    char character3 {'r'};
    char character4 {'o'};
    char character5 {'w'};

    cout << character1 << endl;
    cout << character2 << endl;
    cout << character3 << endl;
    cout << character4 << endl;
    cout << character5 << endl;

    char value {65};
    int value_int = value; // copia o valor numérico para um int

    cout << "value: " << value << endl;
    cout << "value(int): " << value_int << endl;

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/01_exec_05.cpp -o cpp/bin/01_exec_05
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/01_exec_05`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/01_exec_05
```

1.8 Inferência de Tipo com auto

O C++ moderno introduziu a palavra-chave `auto`, que permite ao compilador deduzir o tipo da variável automaticamente com base no valor atribuído. Em vez de escrever o tipo explicitamente, o programador deixa o compilador decidir.

```
auto inteiro {42};    // deduz int
auto decimal {3.14};  // deduz double
```

```
auto real {2.5f};      // deduz float (por causa do sufixo f)
auto caractere {'K'};  // deduz char
```

A dedução acontece uma única vez, no momento da declaração. Depois disso, o tipo é fixo e não muda mais. Se o compilador deduzir `unsigned int`, a variável será `unsigned int` para sempre, mesmo que depois receba um valor negativo.

```
auto contador {100u};    // deduz unsigned int
contador = -5;           // PERIGO: -5 é reinterpretado como unsigned
                        // o valor armazenado será 4294967291
```

Esse é o principal cuidado com `auto`. Como o tipo não aparece escrito no código, o programador pode perder a noção de qual tipo a variável realmente possui, e atribuições incompatíveis podem passar despercebidas sem nenhum erro do compilador.

Quando usamos `auto`, o compilador decide o tipo com base no valor e no sufixo. Sem sufixo, um número inteiro é `int` e um número decimal é `double`. Os sufixos permitem forçar outros tipos. A tabela abaixo resume os mais comuns.

Sufixo	Tipo deduzido	Exemplo
nenhum (inteiro)	<code>int</code>	<code>auto x {10};</code>
nenhum (decimal)	<code>double</code>	<code>auto x {3.14};</code>
<code>f</code>	<code>float</code>	<code>auto x {3.14f};</code>
<code>L</code>	<code>long double</code>	<code>auto x {3.14L};</code>
<code>u</code>	<code>unsigned int</code>	<code>auto x {10u};</code>
<code>ul</code>	<code>unsigned long int</code>	<code>auto x {10ul};</code>
<code>ll</code>	<code>long long int</code>	<code>auto x {10ll};</code>

Observe que `u`, `ul` e `ll` são sufixos para tipos inteiros. O `u` marca o valor como sem sinal (`unsigned int`), o `ul` combina sem sinal com tamanho maior (`unsigned long int`) e o `ll` indica o maior tipo inteiro padrão (`long long int`, 8 bytes). Esses sufixos só se aplicam a números inteiros, nunca a números decimais.

O programa abaixo demonstra a dedução de tipos com `auto`, o uso de sufixos e a verificação do tamanho de cada variável com `sizeof`.

Exemplo 6: Inferência de tipo com auto

```
%%writefile cpp/src/01_exec_06.cpp
#include <iostream>

using namespace std;

int main(){

    auto var1 {12};
    auto var2 {13.0};
    auto var3 {14.0f};
    auto var4 {15.0l};
    auto var5 {'e'};

    // Sufixos de modificadores inteiros

    auto var6 {123u}; // Unsigned int
    auto var7 {123ul}; // Unsigned long int
    auto var8 {123ll}; // Long long int

    cout << "var1 ocupa: " << sizeof(var1) << " bytes" << endl;
    cout << "var2 ocupa: " << sizeof(var2) << " bytes" << endl;
    cout << "var3 ocupa: " << sizeof(var3) << " bytes" << endl;
    cout << "var4 ocupa: " << sizeof(var4) << " bytes" << endl;
    cout << "var5 ocupa: " << sizeof(var5) << " bytes" << endl;
    cout << "var6 ocupa: " << sizeof(var6) << " bytes" << endl;
    cout << "var7 ocupa: " << sizeof(var7) << " bytes" << endl;
    cout << "var8 ocupa: " << sizeof(var8) << " bytes" << endl;

    cout << endl;
    cout << "--- Perigo do auto ---" << endl;

    // O tipo é fixo após a declaração, não muda mais
    auto var9 {333u}; // deduz unsigned int
    cout << "var9 antes: " << var9 << endl;

    var9 = -22; // PERIGO: -22 é reinterpretado como unsigned
    cout << "var9 depois de receber -22: " << var9 << endl;
    // Saída: 4294967274 (e não -22)

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/01_exec_06.cpp -o cpp/bin/01_exec_06
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/01_exec_06`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/01_exec_06
```

✓ 1.9 Constantes

Até agora, todas as variáveis que criamos podem ter seu valor alterado livremente ao longo do programa. Na maioria dos casos isso é desejável, mas imagine o seguinte cenário.

```
double pi {3.14159265358979};
// ... 200 linhas de código depois ...
pi = 2.5; // alguém alterou pi por engano
```

O programa compila sem erro, executa normalmente e produz resultados errados. Encontrar esse tipo de bug é extremamente difícil, pois o código parece correto à primeira vista. O valor de pi simplesmente não deveria ser alterável.

Esse problema aparece sempre que trabalhamos com valores que são fixos por natureza, como constantes matemáticas, limites de configuração, taxas, velocidades físicas. Se esses valores puderem ser alterados acidentalmente, qualquer parte do programa pode introduzir um erro silencioso.

Para resolver isso, o C++ oferece a palavra-chave `const`. Uma variável declarada com `const` é somente leitura. Após a inicialização, qualquer tentativa de alterar seu valor gera um erro de compilação, ou seja, o problema é detectado antes do programa sequer rodar.

```
const double pi {3.14159265358979};
const int max_tentativas {3};

// pi = 2.5;           // ERRO de compilação: não pode alterar const
// max_tentativas = 5; // ERRO de compilação: não pode alterar const
```

Com `const`, o cenário perigoso que descrevemos acima simplesmente não compila. O compilador protege o programador de si mesmo.

A convenção em muitos projetos é usar nomes em letras maiúsculas para constantes, o que facilita identificar visualmente quais valores são fixos ao ler o código.

```
const double GRAVIDADE {9.81};
const int LIMITE_USUARIOS {100};
const double TAXA_IMPOSTO {0.15};
```

Literais. Todo valor fixo escrito diretamente no código é chamado de literal. Quando escrevemos `42`, `3.14`, `'A'` ou `"Texto"`, estamos usando literais. Eles são constantes por natureza, mas não possuem nome. A diferença entre um literal e uma variável `const` é que o literal aparece solto no código sem identificação, enquanto a `const` tem um nome que explica o que aquele valor representa. Compare as duas versões abaixo.

```
double desconto = preco * 0.15;           // o que é 0.15?
double desconto = preco * TAXA_IMPOSTO;   // muito mais claro
```

Na primeira linha, quem lê o código precisa adivinhar o que `0.15` significa. Na segunda, o nome `TAXA_IMPOSTO` torna a intenção óbvia. Esse é um dos principais motivos para usar `const` em vez de espalhar literais pelo código.

A diretiva `#define`. Em código legado, é comum encontrar constantes definidas com `#define`, que é uma instrução do pré-processador herdada da linguagem C.

```
# define PI 3.14159 // forma antiga, evite em C++ moderno
const double PI {3.14159}; // forma moderna, preferível
```

A diferença é que `#define` faz uma simples substituição de texto antes da compilação. O pré-processador troca toda ocorrência de `PI` pelo texto `3.14159`, sem nenhuma verificação de tipo. Se houver um erro, a mensagem do compilador apontará para o número

3.14159 e não para o nome `PI`, o que dificulta a depuração. Já `const` cria uma variável real, com tipo definido, verificada pelo compilador e visível no depurador. Em C++ moderno, sempre prefira `const`.

```
%writefile cpp/src/01_exec_07.cpp
#include <iostream>

using namespace std;

int main(){

    // Constantes com nomes em maiúsculas (convenção)
    const double PI {3.14159265358979};
    const double GRAVIDADE {9.81};
    const double TAXA_IMPOSTO {0.15};

    // Constantes com nomes descritivos
    const int idade {20};
    const float altura {1.67f};

    // Tentativas de alterar constantes geram erro de compilação
    # idade = 17;          // ERRO: não pode alterar const
    # altura = 1.8f;       // ERRO: não pode alterar const
    # PI = 2.5;            // ERRO: não pode alterar const

    // Constantes podem ser usadas em expressões normalmente
    double circunferencia {2 * PI * 10.0};
    int meses_vividos {12 * idade};

    // Literais vs const: compare a clareza
    double preco {250.0};
    double desconto_ruim = preco * 0.15;          // o que é 0.15?
    double desconto_claro = preco * TAXA_IMPOSTO; // muito mais claro

    cout << "PI: " << PI << endl;
    cout << "Gravidade: " << GRAVIDADE << endl;
    cout << "Idade: " << idade << endl;
    cout << "Circunferência: " << circunferencia << endl;
    cout << "Meses vividos: " << meses_vividos << endl;

    cout << endl;

    cout << "Preco: " << preco << endl;
    cout << "Desconto (literal): " << desconto_ruim << endl;
    cout << "Desconto (const): " << desconto_claro << endl;

    return 0;
}
```